Biblioteca UNIMINUTO – Arrays, Métodos Y Dimensiones

Nombre:

Miguel Angel Larrea Calle

Facultad:

Ingeniería de Software

Institución:

Corporación Universitaria Minuto de Dios

Curso:

Lógica de Programación

NRC – 10- 56768

Docente:

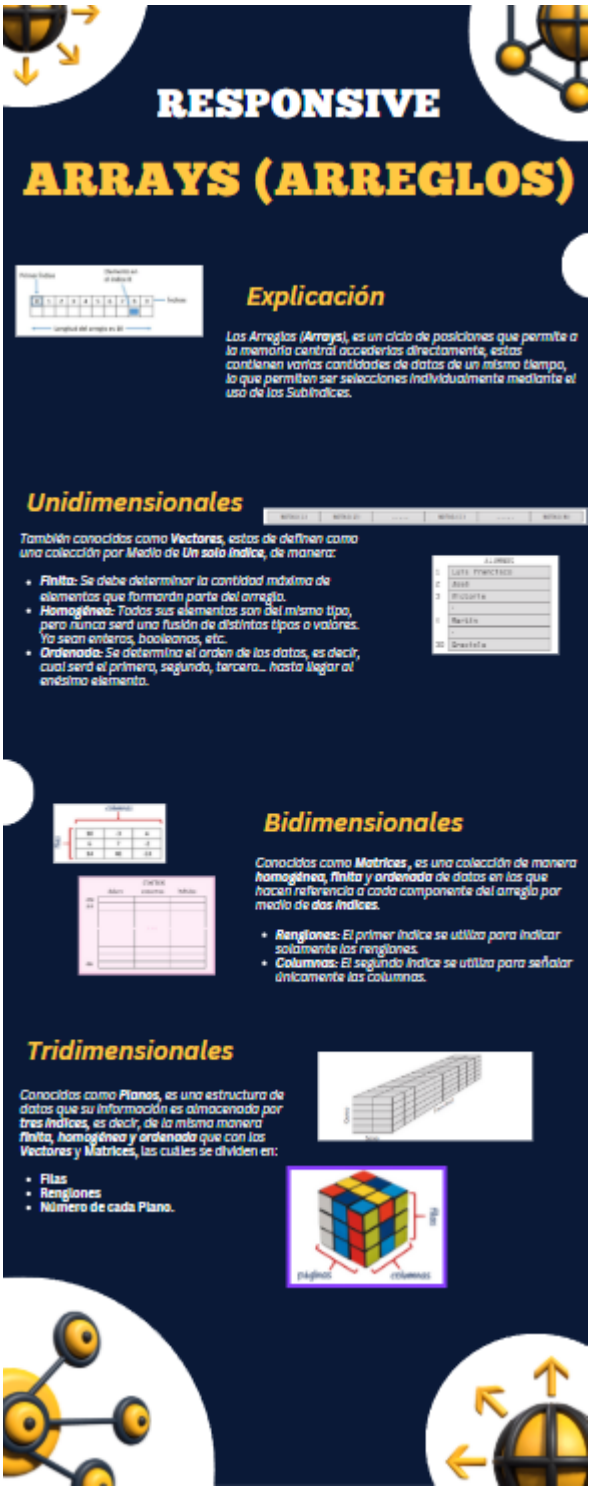
Jose Alirio Barragan Sanchez

Fecha:

18 de Mayo

Infografía

Arreglos [Arrays]



Cuestionario

Array

También conocido como un **Arreglo**, es una estructura que organiza los datos de un mismo tipo en varias posiciones próximas de la memoria, lo cual, permite acceder a cada uno de los elementos directamente por medio de uno o varios índices.

Esta es utilizada para el respectivo uso y almacenamiento de los diversos conjuntos de datos, ya sea también una dimensión (listas) o varias dimensiones.

Unidimensionales

Llamados **Vectores,** es una estructura de datos organizados de manera lineal y accesible por medio de **Un solo índice**, este tipo de arreglo se define por ser de manera **Finita, homogénea** y **Ordenada**. **(**Osvaldo Cairó, 2006**)**

Ejemplo:

Asignar Y Guardar las Calificaciones de los Estudiantes En Cada Respectiva Posición, dependiendo del Orden.

Bidimensionales

Su nombre comúnmente conocido como **Matrices,** es una estructura de datos manejada por medio de **Dos índices,** las cuales permiten organizar en **Filas** y **Columnas**. De tal manera que en uno permita identificar la **Fila** y en el otro la **Columna** respectivamente. **(**Osvaldo Cairó, 2006**)**

Al igual que los **Vectores,** estas son definidas de manera **Finita, homogénea** y **Ordenada.**

Ejemplo:

Asignar, Guardar y Almacenar las Calificaciones de Varios Estudiantes En distintas Asignaturas.

Filas = Estudiantes

Columnas = Materias.

Métodos de Ordenamiento En Un Vector

1. **Burbuja (Bubble Sort):** Este método ordena los elementos comparando cada par adyacente y cambiándolos de posición si no están en el orden correcto. Este proceso se repite hasta que todo el vector está correctamente ordenado.

2. **Burbuja mejorada:** Es una versión optimizada del método de burbuja que incluye una verificación adicional: si durante una pasada completa no se realiza ningún intercambio, el algoritmo finaliza anticipadamente, lo cual mejora su eficiencia en ciertos casos.

3. **Selección (Selection Sort):** Este algoritmo busca el elemento más pequeño entre los no ordenados y lo intercambia con el que ocupa la primera posición desordenada. Repite este procedimiento hasta que todo el vector esté ordenado.

4. **Inserción (Insertion Sort):** Recorre el vector tomando un elemento a la vez y lo inserta en la posición adecuada dentro de los elementos previamente ordenados, desplazando los necesarios. Es útil cuando la lista está parcialmente ordenada.

5. **Shell (Shell Sort):** Mejora el ordenamiento por inserción al comparar elementos que están separados por una cierta distancia. Esta distancia se va reduciendo en cada iteración hasta llegar a comparaciones entre elementos consecutivos, permitiendo ordenar más eficientemente.

6. **Por Fusión (Merge Sort):** Este algoritmo divide el vector en mitades de forma recursiva, ordena cada mitad por separado y luego las combina en una sola lista ordenada. Su eficiencia es notable en listas grandes.

7. **Ordenamiento rápido (Quick Sort):** Selecciona un elemento como pivote y reorganiza el vector colocando los menores que el pivote a su izquierda y los mayores a su derecha. Luego repite este proceso recursivamente en las sublistas. Hay varias formas de elegir el pivote (inicio, fin, aleatorio o medio).

Búsqueda Binaria

El Algoritmo de búsqueda binaria es un método eficiente utilizado para localizar un elemento dentro de una lista que ya se encuentra ordenada.

Su Funcionamiento se basa en **dividir** repetidamente la lista a la mitad, comparando el elemento buscado con el valor central en cada iteración, lo que permite **Reducir** significativamente el número de comparaciones necesarias hasta **Encontrar** la posición del elemento o **determinar** que no se encuentra en la lista.

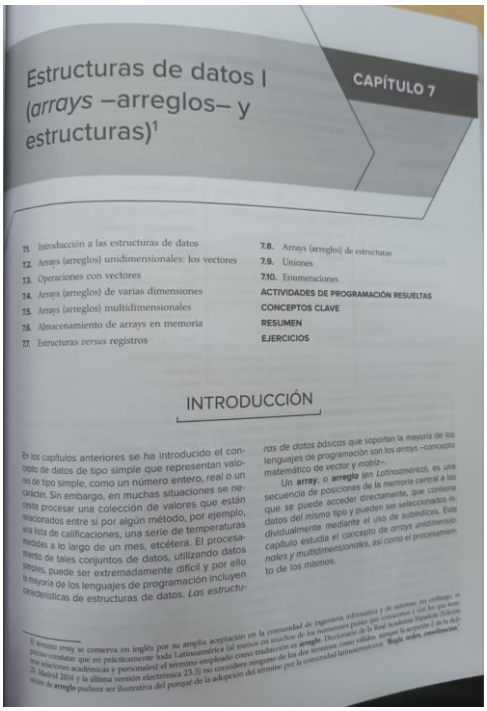
Tridimensionales

También llamado **Plano**, es una estructura de datos que organiza la información utilizando **Tres índices**, Uno para las **Filas**, Otro para los **Renglones** y El Último para identificar **Cada Plano dentro del Conjunto**. Al igual que los Vectores y Matrices, estos arreglos son **Finitos**, **homogéneos** y **Ordenados**.

Estos son Aplicados y Útiles, especialmente en aplicaciones donde se necesita representar datos en un espacio tridimensional, Comúnmente En Procesamientos En Modelado de Objetos En Entornos virtuales.

Imágenes

- **Fundamentos de Programación Biblioteca - (Presencial)**



Estructuras de datos I (*arrays* –arreglos– y estructuras)[1]

CAPÍTULO 7

7.1. Introducción a las estructuras de datos
7.2. Arrays (arreglos) unidimensionales: los vectores
7.3. Operaciones con vectores
7.4. Arrays (arreglos) de varias dimensiones
7.5. Arrays (arreglos) multidimensionales
7.6. Almacenamiento de arrays en memoria
7.7. Estructuras *versus* registros
7.8. Arrays (arreglos) de estructuras
7.9. Uniones
7.10. Enumeraciones
ACTIVIDADES DE PROGRAMACIÓN RESUELTAS
CONCEPTOS CLAVE
RESUMEN
EJERCICIOS

INTRODUCCIÓN

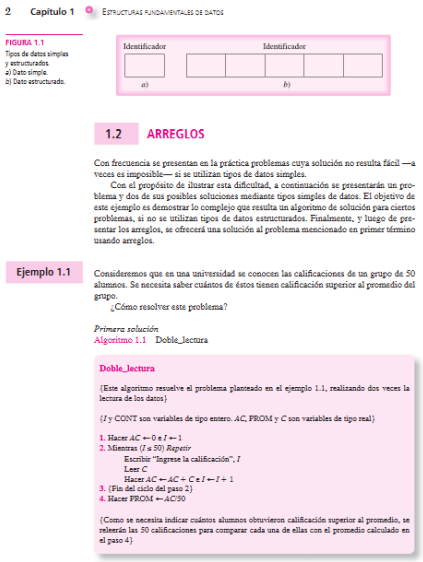
En los capítulos anteriores se ha introducido el concepto de datos de tipo simple que representan valores de tipo simple, como un número entero, real o un carácter. Sin embargo, en muchas situaciones se necesita procesar una colección de valores que están relacionados entre sí por algún método, por ejemplo, una lista de calificaciones, una serie de temperaturas medias a lo largo de un mes, etcétera. El procesamiento de tales conjuntos de datos, utilizando datos simples, puede ser extremadamente difícil y por ello la mayoría de los lenguajes de programación incluyen características de estructuras de datos. *Las estructuras de datos básicas* que soportan la mayoría de los lenguajes de programación son los *arrays* –concepto matemático de vector y matriz–.

Un **array**, o **arreglo** (en *Latinoamérica*), es una secuencia de posiciones de la memoria central a las que se puede acceder directamente, que contiene datos del mismo tipo y pueden ser seleccionados individualmente mediante el uso de subíndices. Este capítulo estudia el concepto de arrays *unidimensionales y multidimensionales*, así como el procesamiento de los mismos.

[1] El término array se conserva en inglés por su amplia aceptación en la comunidad de ingeniería informática y de sistemas, sin embargo, se puede constatar que en prácticamente toda Latinoamérica (al menos en muchos de los numerosos países que conforman) con las que tenemos relaciones académicas y personales) el término empleado como traducción es **arreglo**. Diccionario de la Real Academia Española (Edición 23. Madrid 2014 y la última versión electrónica 23.3) no considera ninguno de los dos términos como válido, aunque la acepción 2 de la definición de **arreglo** pudiera ser ilustrativa del porque de la adopción del término por la comunidad latinoamericana: **Regla, orden, coordinación**

Explicación: Gran Parte del Documento Y Sus Explicaciones Sobre **Arrays**, Fue Sacado A base del Libro Recomendado Por El Profesor llamado **Fundamentos de Programación,** En Este Caso, de Manera **Presencial** En la **biblioteca** de la **UNIMINUTO**.

- **Estructuras de Datos Biblioteca - (Virtual)**



2 Capítulo 1 ESTRUCTURAS FUNDAMENTALES DE DATOS

FIGURA 1.1
Tipos de datos simples y estructurados.
a) Dato simple.
b) Dato estructurado.

Identificador
a)
Identificador
b)

1.2 ARREGLOS

Con frecuencia se presentan en la práctica problemas cuya solución no resulta fácil —a veces es imposible— si se utilizan tipos de datos simples.

Con el propósito de ilustrar esta dificultad, a continuación se presentarán un problema y dos de sus posibles soluciones mediante tipos simples de datos. El objetivo de este ejemplo es demostrar lo complejo que resulta un algoritmo de solución para ciertos problemas, si no se utilizan tipos de datos estructurados. Finalmente, y luego de presentar los arreglos, se ofrecerá una solución al problema mencionado en primer término usando arreglos.

Ejemplo 1.1

Consideremos que en una universidad se conocen las calificaciones de un grupo de 50 alumnos. Se necesita saber cuántos de éstos tienen calificación superior al promedio del grupo.

¿Cómo resolver este problema?

Primera solución
Algoritmo 1.1 Doble_lectura

Doble_lectura

{Este algoritmo resuelve el problema planteado en el ejemplo 1.1, realizando dos veces la lectura de los datos}

{*I* y CONT son variables de tipo entero. *AC*, PROM y *C* son variables de tipo real}

1. Hacer $AC \leftarrow 0$ e $I \leftarrow 1$
2. Mientras ($I \leq 50$) *Repetir*
   Escribir "Ingrese la calificación", *I*
   Leer *C*
   Hacer $AC \leftarrow AC + C$ e $I \leftarrow I + 1$
3. {Fin del ciclo del paso 2}
4. Hacer PROM $\leftarrow AC/50$

{Como se necesita indicar cuántos alumnos obtuvieron calificación superior al promedio, se leerán las 50 calificaciones para comparar cada una de ellas con el promedio calculado en el paso 4}

Explicación: Acerca Sobre los Arreglos **Uni, bidi** y **Tri** dimensionales, Fue Realizado También Por Medio de Un Libro Electrónico En **McGraw – Hill**, llamado **Estructuras de Datos**, En Este Caso, de Manera **Virtual** En los **Libros Electronicos** de la **UNIMINUTO.**

Bibliografía

(Referencias)

- Joyanes Aguilar, L. (2020). *Fundamentos de programación: algoritmos, estructura de datos y objetos*. McGraw-Hill.

https://www-ebooks7-24-com.ezproxy.uniminuto.edu/?il=10409

- Cairó Battistutti, O., Guardati, S.(2006). *Estructuras de datos*. McGraw-Hill. https://www-ebooks7-24-com.ezproxy.uniminuto.edu/?il=5339

- Gbaudino. (2023). *Métodos de ordenamiento* [Repositorio GitHub]. GitHub. https://github.com/gbaudino/MetodosDeOrdenamiento?tab=readme-ov-file#8---m%C3%A9todo-de-ordenamiento-del-peine-combsort

- Khan Academy. (s.f.). *Búsqueda binaria*. https://es.khanacademy.org/computing/computer-science/algorithms/binary-search/a/binary-search#:~:text=La%20b%C3%BAsqueda%20binaria%20es%20un,ubicaciones%20posibles%20a%20solo%20una